

AdaptiveDMA SoC 프로젝트 보고서

CPU latency feedback 을 이용한 DMA 대역폭 제어

Arty Z7-20 · Zynq-7000 XC7Z020 · Vivado/Vitis 2022.2

요약

공유 DDR 에서 CPU 와 DMA 가 동시에 동작할 때 발생하는 CPU 지연을 줄이기 위해 Adaptive RBR 을 구현했다. 논문의 Monitor-Throttler 구조를 AXI4 DMA 에 적용하고, CPU0 의 latency 측정값에 따라 DMA 대역폭 제한을 자동으로 강화하거나 완화하는 hardware controller 를 추가했다. 25 개 설정의 보드 측정으로 제어 단계를 선정하고, CPU1 의 DDR 부하를 변화시키며 Fixed RBR 과 성능을 비교했다.

최종 실험에서 Adaptive RBR 은 부하에 따라 50%와 66.7% 단계를 선택했다. DMA 처리량은 Fixed RBR 50%의 381.46 MiB/s 에서 426.12 MiB/s 로 11.71% 증가했고, CPU latency 상한 초과율은 0%에서 5.67%로 변했다. 부하가 낮을 때 DMA 대역폭을 더 사용하는 효과와 설정 전환에 따른 CPU 지연을 함께 평가했다.

1 배경과 목표

1.1 CPU 와 DMA 의 공유 DDR 경합

Zynq 의 CPU 와 PL DMA 는 같은 DDR controller 를 통해 메모리에 접근한다. DMA 가 많은 요청을 연속으로 보내면 CPU 의 메모리 요청도 영향을 받는다. 본 프로젝트의 초기 실험에서는 DMA 를 최대로 동작시켰을 때 CPU 의 평균 access 시간이 약 두 배로 증가했다.

DMA 를 낮은 대역폭으로 제한하면 CPU 지연을 줄일 수 있다. 반면 같은 제한을 계속 유지하면 다른 부하가 줄어든 뒤에도 DMA 속도가 낮게 유지된다. 따라서 CPU 의 지연을 관찰하며 DMA 제한을 강화하거나 완화하는 제어가 필요하다.

이 프로젝트의 목표는 CPU latency 를 설정한 상한 이내로 제어하면서, 낮은 비율로 고정한 DMA 보다 더 높은 처리량을 확보하는 것이다. 평가는 CPU latency, 상한 초과율, DMA 처리량과 실제 설정 변화로 구성했다.

1.2 RBR 기반의 CPU feedback 제어

Valente 등의 연구[1]는 전송 시간을 측정하는 Monitor 와 대기 시간을 삽입하는 Throttler 를 결합한 Runtime Bandwidth Regulator(RBR)를 제안한다. 또한 상위 제어기가 실행 중 목표 대역폭 비율을 변경하는 구조를 설명한다.

본 프로젝트는 RBR 의 측정·대기 계산을 구현하고, CPU latency sample 에 따라 목표 비율을 자동으로 선택하는 정책을 추가했다. 이를 위해 AXI4 DMA, 설정 레지스터, RBR, Adaptive Controller 와 CPU 측정 프로그램을 통합했다. Fixed RBR 을 기준으로 설정별 성능을 측정한 뒤, CPU1 의 부하가 변하는 조건에서 Adaptive 동작을 평가했다.

2 시스템 설계

2.1 데이터 경로와 CPU feedback 경로

DATA PATH



FEEDBACK

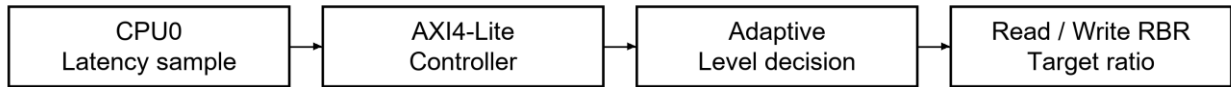


그림 1 R/W 데이터 경로와 CPU latency feedback 경로

RDMA 가 DDR 의 source 를 읽고, 깊이 64 의 FIFO 를 거쳐 WDMA 가 destination 에 기록한다. 데이터 폭은 64 bit 이며 한 beat 는 8 byte 다. CPU0 는 AXI4-Lite 로 DMA 명령과 설정을 전달한다. DMA 의 AXI4 master 는 PS 의 HP0 port 를 통해 DDR 에 접근한다.

DMA 는 burst 1/2/4/8/16 beat 와 outstanding 1/2/4 를 지원한다. Burst 는 한 요청에서 옮길 beat 수이고, outstanding 은 주소 요청이 수락된 뒤 응답 완료를 기다리는 transaction 수다. 이 두 설정으로 전송 효율과 CPU 지연의 관계를 측정했다. DMA 는 4 KiB 경계를 넘는 전송을 나누어 요청한다.

2.2 Monitor 와 Throttler

Read 와 Write 경로에는 각각 RBR 을 둔다. Monitor 는 threshold byte 가 전송되는 데 걸린 cycle 수 copycc 를 측정한다. Read 는 수신한 byte 를, Write 는 WSTRB 가 지정한 유효 byte 를 센다. Throttler 는 이 결과로 추가 대기 시간 idlecc 를 계산한다.

$$\delta = (100 - \text{THR}) / \text{THR}$$

$$\text{idlecc} = \max((1 + \delta) \times \text{nomcc} - \text{copycc}, 0)$$

THR 은 목표 대역폭 비율이고, nomcc 는 threshold 를 매 cycle 연속 전송할 때 필요한 기준 cycle 수다. 예를 들어 threshold 256 byte 는 32 cycle 에 해당한다. 목표 50%에서 copycc 가 32 이면 32 cycle 을 추가로 기다린다. 메모리 응답 지연으로 copycc 가 40 이 되면 추가 대기는 24 cycle 로 줄어든다. 이미 발생한 지연까지 포함해 전송과 대기의 총 시간을 조절하는 식이다.

Regulator 는 Monitor 와 Throttler 를 연결해 데이터 전송과 대기를 반복한다. Read 에서는 RREADY 로 데이터 수신을 늦추고, Write 에서는 WVALID 로 새 데이터 제시를 늦춘다. 내부 DMA 에도 같은 대기를 전달해 데이터가 실제로 전송될 때만 다음 beat 로 진행하도록 했다. δ 는 Q8.8 fixed-point 로 저장한다.

WAIT 가 끝나면 남은 데이터를 이어서 전송한다. AR/AW 는 DMA 의 outstanding 제한에 따라 발행되고, 완료된 write 의 B response 는 계속 수신한다. FIFO 는 Read 와 Write 의 속도 차이를 흡수하며, 가득 차면 Read 쪽으로 backpressure 를 전달한다.

2.3 Adaptive Controller

CPU0 는 DDR access latency 를 측정해 LAT_SAMPLE 레지스터에 기록한다. Adaptive Controller 는 새 sample 이 들어올 때 현재 RBR 단계를 유지하거나 한 단계 변경한다. CPU 는 측정과 sample 전달을 맡고, PL 은 단계 선택과 데이터 전송 제어를 수행한다.

최종 제어에서는 burst 16, outstanding 4, threshold 256 byte 와 nominal 32 cycle 을 고정하고 Read/Write RBR 비율을 함께 변경한다. 각 설정을 선택한 측정 근거는 5 절에 정리했다. 표 1 의 WAIT 는 copycc 가 32 cycle 인 경우다.

표 1 Adaptive Controller 의 RBR 단계

단계	목표 비율	δ 의 Q8.8 값	기준 WAIT
L0	50%	256	32 cycle
L1	66.7%	128	16 cycle
L2	80%	64	8 cycle
L3	100%	0	0 cycle

목표 비율의 기준은 64-bit data channel 이 매 cycle 전송할 때의 대역폭이다. PL clock 100 MHz 에서 이 값은 800 MB/s, 즉 762.94 MiB/s 다. 실제 처리량은 RBR 비율과 DDR 응답·backpressure 의 영향을 받는다. L3 에서는 추가 WAIT 가 0 이 된다.

2.4 단계 전환과 반복 전환 억제

최종 실험의 low/high limit 은 CPU-only 평균의 130%와 150%다. High limit 을 넘으면 제한을 강화하고, low limit 보다 낮으면 제한을 완화한다. 두 값 사이에서는 현재 단계를 유지한다. 이 hysteresis band 가 한 경계 부근에서 상승과 하락이 반복되는 것을 줄인다.

High limit 초과를 두 번 연속 확인하면 한 단계 낮추고, low limit 미만을 세 번 연속 확인하면 한 단계 높인다. 이 confirmation count 로 일시적인 변화에 대한 반응을 줄이고, CPU 보호 방향이 더 빨리 결정되도록 했다. 단계 변경 후에는 다음 네 sample 동안 유지하는 hold 를 적용한다. Hold 가 끝나면 연속 횟수를 다시 센다.

Sample 한 개는 CPU 의 16,384 회 메모리 접근에 대한 평균값이다. Controller 의 단계 범위는 L0~L3 이며, 보드 실험은 각 반복을 L1 에서 시작했다. 새 비율은 다음 RBR 측정 window 부터 적용한다. 현재 window 는 처음 저장한 설정으로 마쳐, 측정 시간과 대기 계산에 같은 설정이 사용되도록 했다.

3 기능 검증과 FPGA 구현

3.1 시뮬레이션 결과

SystemVerilog testbench 에서 데이터 일치, 4 KiB 경계, backpressure 와 실행 중 설정 변경을 확인했다. DMA/RBR 의 35 개 test 는 오류 0 으로 통과했다. Adaptive 통합 test 에서도 한 DMA command 안에서 L0~L3 를 전환하며 설정 적용과 데이터 일치를 확인했다. 이 결과를 바탕으로 FPGA 구현과 보드 측정을 진행했다.

3.2 Critical path 개선

Configurable WDMA 에서 burst 크기를 결정하고 다음 주소를 계산하는 조합 경로가 길어 setup timing 을 위반했다. Beat 수를 다시 byte 수로 환산하는 과정을 줄이고, 이미 계산한 전송 byte 수로 주소를 갱신했다. 전체 configurable DMA 의 post-synthesis WNS 는 -0.364 ns 에서 +0.461 ns 로 개선되었다.

RBR 통합 과정에서는 Monitor 를 remaining-byte counter 로 바꾸고 WLAST 와 마지막 WSTRB 조건을 미리 계산했다. 해당 버전의 post-synthesis critical path delay 는 11.386 ns 에서 8.935 ns 로 줄었다. 수정 후 기존 test 를 다시 실행해 기능을 확인했다.

3.3 최종 구현 결과

최종 설계는 PL clock 100 MHz 에서 배치·배선을 완료했다. Post-route WNS 는 +0.047 ns, WHS 는 +0.030 ns 로 setup·hold timing 을 만족했다. Setup slack 이 가장 작은 경로는 Controller 의 Adaptive enable register 에서 Write Throttler 의 목표 cycle 계산 register 까지 이어지며, Q8.8 곱셈·덧셈을 포함한 data path delay 는 9.814 ns 다.

표 2 최종 전체 PL 의 자원 사용량 Post-place 기준

자원	사용량	가용량	사용률
Slice LUT	2,646	53,200	4.97%
Flip-Flop	2,388	106,400	2.24%
BRAM tile	0	140	0.00%
DSP	2	220	0.91%

표 2 는 DMA IP 와 AXI 연결 회로를 포함한다. Vivado 의 전체 SoC on-chip power 추정값은 1.414 W 이며, vectorless switching 조건에서 confidence 는 Medium 이다.

4 보드 실험 방법

4.1 CPU DDR latency 측정

CPU0 에 1 MiB DDR buffer 를 마련하고 64-byte node 16,384 개로 나누었다. 각 node 의 첫 word 에는 다음에 접근할 node 번호를 저장한다. CPU 는 그 word 를 읽은 뒤 다음 주소를 정한다. 앞선 load 결과가 다음 load 주소를 결정하는 dependent pointer chase 로 CPU 의 메모리 대기 시간을 측정했다.

측정 buffer 는 Normal Non-cacheable 로 설정했다. 1,024 회 warm-up 후 XTime_GetTime 으로 16,384 회 접근 직전과 직후의 시간을 읽는다. 두 timestamp 의 차이에서 연속된 timer 읽기로 측정한 overhead 를 빼고, CPU clock 기준으로 환산한다.

$$\text{CPU cycles/access} = \text{보정한 timer ticks} \times (\text{CPU clock} / \text{timer clock}) / 16,384$$

모든 mode 는 같은 pointer-chase loop 와 -O2 빌드를 사용한다. 측정값에는 동일한 loop 실행 비용이 포함된다. UART 출력과 DMA register 확인은 시간 측정 전후에 수행한다. Slowdown 은 DMA 동시 실행으로 CPU access 시간이 늘어난 비율이다.

$$\text{Slowdown (\%)} = (\text{동시 실행 latency} - \text{CPU-only latency}) / \text{CPU-only latency} \times 100$$

4.2 DMA 동시 실행과 데이터 확인

DMA source, destination 과 CPU 측정 buffer 는 서로 다른 DDR 영역에 둔다. DMA 가 8 MiB copy 를 시작한 뒤 busy 를 확인하고 CPU 시간을 측정한다. 측정 직후에도 busy 가 유지된 sample 을 기록한다. 그 사이 DMA 가 완료되었으면 새 command 를 시작해 다시 측정한다.

완료된 command 마다 Read/Write beat 수와 transaction 수를 확인하고, destination 양 끝과 command 마다 달라지는 두 위치를 source 와 비교한다. 각 반복의 마지막에는 전체 8 MiB 도 비교한다. WAIT 와 window counter 를 함께 저장해 RBR 동작을 확인한다.

4.3 CPU1 동적 부하와 실행 순서

CPU1 은 별도 8 MiB buffer 를 반복해서 읽고 쓰는 load generator 를 실행한다. CPU0 가 shared memory 에 ON 을 쓰면 접근을 계속하고, OFF 를 쓰면 대기한다. 이를 통해 같은 DMA 와 CPU0 측정 프로그램에 추가 DDR 부하가 들어오고 빠지는 조건을 만든다. Adaptive Controller 는 CPU0 의 latency sample 을 입력받아 동작한다.

최종 실험의 각 mode 는 OFF → ON → OFF → ON 순서로 구간당 25 개 sample 을 수집한다. 이 과정을 세 번 반복해 mode 당 300 개 sample 을 얻었다. CPU_ONLY 를 먼저 실행하고, 그중 OFF 구간의 평균에 1.3 과 1.5 를 곱해 모든 비교 mode 에 같은 low/high limit 을 적용했다. 각 DMA command 는 하나의 ON 또는 OFF 구간 안에서 완료시켰다.

4.4 집계 지표

Mean, P95, P99 는 sample 별 평균 access latency 에서 계산한다. P95 와 P99 는 sample 을 오름차순으로 정렬했을 때 각각 95%와 99% 위치의 값이다. 상한 초과율은 high limit 을 넘은 sample 수를 전체 sample 수로 나눈 값이다. DMA 처리량은 완료한 copy byte 수를 누적 DMA busy 시간으로 나눈다. 각 실험은 CPU-only 기준을 다시 측정하고 같은 실행의 mode 끼리 비교했다.

5 측정 결과와 설정 선택

5.1 Fixed RBR 동작 확인

첫 경합 실험은 CPU_ONLY, DMA_MAX, RBR_100, RBR_FIXED 50%를 30 회씩 실행했다. CPU 평균 latency 는 CPU_ONLY 100.62 cycles/access 에서 DMA_MAX 206.83 으로 증가했다. RBR 50%를 적용하면 CPU latency 는 125.28 로 낮아지고, DMA 처리량은 734.25 MiB/s 에서 381.46 MiB/s 로 감소했다.

RBR 100%는 CPU 평균 206.79 cycles/access 와 DMA 처리량 734.28 MiB/s 를 기록해 bypass 와 거의 같은 결과를 보였다. 추가 WAIT 가 없는 설정의 성능을 확인한 뒤, RBR 비율과 DMA 설정을 바꾸는 Sweep 을 수행했다.

5.2 25 개 설정의 Parameter sweep

Adaptive Controller 의 제어 변수와 단계별 설정을 고르기 위해 25 개 조건을 비교했다. Burst 5 종과 outstanding 3 종의 조합 15 개, RBR 비율 6 개, threshold 추가 2 개, Read-only 와 Write-only 2 개를 각각 100 회 측정했다. 설정별 CPU-only 평균은 106.34~106.39 cycles/access 였으며, 상한은 해당 평균의 150%로 두었다.

Burst 16 에서 outstanding 1 → 2 → 4 의 DMA 처리량은 238.25 → 487.12 → 734.23 MiB/s, CPU 평균은 120.10 → 133.86 → 208.34 cycles/access 였다. 여러 요청을 동시에 진행하면 DMA 처리량이 증가하지만 CPU 지연도 커지는 구간이 확인되었다.

표 3 RBR 비율별 Sweep Burst 16 Outstanding 4 Threshold 256 byte

목표 비율	CPU mean cycles/access	CPU P99 cycles/access	DMA MiB/s
25%	117.32	120.80	190.73
33.3%	120.23	122.97	254.31
50%	128.32	133.91	381.46
66.7%	136.76	150.31	508.61
80%	151.15	151.49	608.87
100%	208.34	209.29	734.22

RBR 50%에서 threshold 256/512/1,536 byte 의 CPU 평균은 128.32/131.58/137.99 cycles/access 였다. DMA 처리량은 모두 약 381.46 MiB/s 였다. 같은 평균 비율에서도 threshold 가 커지면 연속 전송과 WAIT 구간이 함께 길어진다. 측정한 세 설정 중 CPU 평균과 P99 가 가장 낮았던 256 byte 를 선택했다.

Read-only, Write-only 와 양쪽 RBR 의 처리량은 약 381.46 MiB/s 로 같았고, Write-only 의 CPU 평균이 123.97 로 가장 낮았다. WDMA 의 대기는 FIFO 를 통해 RDMA 에도 전달되므로 한쪽 RBR 만으로도 전체 copy 속도가 제한될 수 있다. 최종 구조는 Read/Write 비율을 각각 설정할 수 있는 양쪽 RBR 을 유지했다.

5.3 Adaptive 단계 선정

초기 후보는 CPU 지연이 낮은 burst 2 와 Write-only RBR 50%, 처리량이 높은 양쪽 RBR 80%와 최대 DMA 설정이었다. 이 구성은 단계가 바뀔 때 burst 길이, 제어 채널과 대역폭 비율이 함께 달라졌다. 최종 설계는 burst 16·outstanding 4 와 양쪽 RBR 을 유지하고, 제어 출력을 RBR 비율로 통일했다. CPU 지연에 따라 조절할 값을 DMA 의 목표 대역폭으로 직접 지정한 것이다.

Sweep 에서 50%는 CPU 평균 128.32 cycles/access 와 381.46 MiB/s 로 지연을 낮추는 보호 단계에 해당했다. 66.7%와 80%는 각각 508.61 과 608.87 MiB/s 를 기록하며 이 실험의 상한 이내에서 처리량을 높였다. 100%는 추가 WAIT 가 없는 단계다. 네 설정을 이용해 CPU 지연이 낮으면 대역폭을 늘리고, 높으면 보호 단계로 돌아가도록 했다.

5.4 동적 부하의 최종 7 개 Mode 비교

CPU1 부하를 변화시키며 총 2,100 개 sample 을 얻었다. CPU_ONLY 의 OFF 구간 평균은 95.04 cycles/access 였고, low/high limit 은 123.55/142.56 이었다. 표 4 의 CPU_ONLY 평균은 ON/OFF 구간을 모두 포함한 값이다.

Fixed maximum 은 burst 16·outstanding 4 와 RBR bypass, Static burst 8 은 burst 8·outstanding 4 와 bypass 를 사용한다. 세 Fixed RBR 과 Adaptive 는 burst 16·outstanding 4 에서 동작한다. 모든 mode 에 동일한 CPU1 부하 순서를 적용했다.

표 4 최종 동적 부하 실험 High limit 142.56 cycles/access 각 300 개 sample

Mode	CPU mean cycles/access	CPU P95 cycles/access	상한 초과율	DMA MiB/s
CPU only	96.28	97.74	0.00%	—
Fixed maximum	230.02	242.18	100.00%	689.69
Static burst 8	149.38	158.76	50.00%	561.66
Fixed RBR 50%	127.97	135.24	0.00%	381.46
Fixed RBR 66.7%	145.69	154.39	56.67%	508.45
Fixed RBR 80%	160.74	177.58	100.00%	606.71
Adaptive RBR	133.43	147.73	5.67%	426.12

Fixed RBR 50%는 300 개 sample 모두 상한 이내였고 처리량은 381.46 MiB/s 였다. 66.7%와 80%는 처리량이 증가하면서 상한 초과율도 높아졌다. Adaptive 는 426.12 MiB/s 로 Fixed 50%보다 11.71% 높은 처리량을 얻었고, 17 개 sample 에서 상한을 넘었다.

동적 부하에서는 같은 66.7% 설정도 상한 초과율이 56.67%에 이르렀다. Sweep 에서 선정한 고정 비율 하나를 계속 쓰는 대신, 현재 CPU latency 를 다시 측정해 제한을 조절해야 하는 이유를 보여준다. 모든 mode 의 데이터·counter 검사와 반복 종료 후 전체 destination 비교는 통과했다.

5.5 부하 변화에 따른 Adaptive 응답



그림 2 CPU latency 와 sample 반영 후 단계. 회색은 CPU1 ON, 점선은 각 반복의 재시작 지점이다.

CPU0 가 측정한 latency 에 따라 Adaptive Controller 는 CPU1 OFF 구간에서 주로 L1(66.7%), ON 구간에서 주로 L0(50%)를 선택했다. OFF 의 150 개 sample 중 101 개가 L1, ON 의 150 개 중 147 개가 L0 였다. Sample 입력에 따른 단계 변경은 상승 5 회와 하락 8 회로 총 13 회였다.

이에 따라 Adaptive DMA 처리량은 OFF 구간 467.49 MiB/s, ON 구간 385.78 MiB/s 였다. Fixed RBR 50%의 처리량은 두 조건 모두 약 381.46 MiB/s 였다. Adaptive 는 부하가 낮을 때 제한을 완화하고, 부하가 높아지면 50% 단계로 돌아와 전송량을 줄였다.

상한 초과와 제어 응답

상한 초과 17 개 중 OFF 구간이 11 개, ON 구간이 6 개였다. OFF 상태에서도 66.7% 단계로 올린 뒤 latency 가 다시 높아져 50%로 돌아오는 전환이 관찰된다. 따라서 외부 부하 변화에 대한 반응과 단계 자체의 영향이 함께 나타났다.

5.6 Burst Adaptive 비교

추가 실험에서는 burst 길이를 2/4/8/16 beat 로 바꾸는 Burst Adaptive 와 RBR 비율을 바꾸는 Adaptive 를 비교했다. Outstanding 4, low/high limit, confirmation 과 hold 는 동일하게 두었다. Burst 단계는 기존 DMA 가 지원하고 Sweep 에서 측정한 네 길이로 구성했다.

이 실험의 high limit 은 CPU-only OFF 평균 100.63 cycles/access 의 150%인 150.94 였다. 같은 실험의 Fixed RBR 50%를 기준으로 비교했다.

표 5 Burst 제어와 RBR 제어 비교 각 300 개 sample

Mode	CPU P95 cycles/access	CPU P99 cycles/access	상한 초과율	DMA MiB/s
Fixed RBR 50%	139.82	140.42	0.00%	381.46
Burst Adaptive	158.42	158.67	22.00%	499.98
RBR Adaptive	149.60	162.21	4.00%	439.47

이 네 단계 구성에서 RBR Adaptive 는 상한 초과율과 P95 가 낮았고, Burst Adaptive 는 DMA 처리량과 P99 가 더 좋았다. RBR 은 burst 길이를 유지하면서 목표 대역폭 비율을 직접 지정할 수 있어 본 설계의 제어 변수로 사용했다. 표 5 의 비교 범위는 위의 네 단계와 공통 전환 정책이다.

6 결론

논문 기반 RBR 에 CPU latency feedback 을 이용하는 Adaptive Controller 를 추가하고, AXI4 DMA 와 함께 Zynq SoC 에 구현했다. 25 개 설정 실험에서 outstanding 증가에 따른 CPU 지연, RBR 비율별 처리량과 threshold 의 영향을 확인했다. 그 결과를 바탕으로 burst·outstanding·threshold 를 고정하고 RBR 비율을 조절하는 구조를 선택했다.

CPU1 의 부하 변화에 따라 RBR 단계가 50%와 66.7% 사이에서 변경되었고, Fixed RBR 50% 대비 DMA 처리량이 11.71% 증가했다. CPU latency 상한 초과율은 5.67%였다. 부하 변화에 따른 자동 조절을 확인했으며, 상한 초과를 줄이기 위한 다음 과제는 feedback 전달 주기와 confirmation·hold 정책의 조정이다.

참고문헌과 실험 자료

[1] G. Valente et al., "Fine-Grained QoS Control via Tightly-Coupled Bandwidth Monitoring and Regulation for FPGA-Based Heterogeneous SoCs," IEEE Transactions on Parallel and Distributed Systems, vol. 36, no. 2, pp. 326–340, 2025. DOI: 10.1109/TPDS.2024.3513416.

원본 측정 로그와 집계값은 results/sweep, results/final, results/burst 에 있다. RTL 과 software 는 HW 와 SW, 검증 코드와 로그는 sim, 최종 FPGA 구현 결과는 docs/implementation 에 정리했다.